

Cloud-to-Edge Integration

INSISN – Integração de Sistemas e Serviços na Nuvem

António Barros

Departamento de Engenharia Informática

From Centralised Cloud to the Cloud-Fog-Edge Continuum

Architecture, Models, and Technological Trade-offs

Introduction

Where Should the Code Run?

- Computing architecture has evolved around distinct operational needs.
- Two classical paradigms represent opposite ends of the spectrum:
 - ▶ Server-centric computing.
 - ▶ Embedded systems.
- Each paradigm addresses specific constraints regarding performance, location, and control.

Server Computing

- Focused on high-performance raw processing power and massive data throughput.
- Key characteristics:
 - ▶ Centralised infrastructure.
 - ▶ High computational capacity and scalable resources.
 - ▶ Handles large-scale data analytics, complex algorithms, and concurrent requests.
- Primary constraint: Requires reliable connectivity to transfer data from the source to the server.

Embedded Systems

- Focused on localised control directly where data is generated.
- Key characteristics:
 - ▶ Tightly integrated with physical hardware and actuators.
 - ▶ Dedicated functionality designed for specific real-time tasks.
 - ▶ Low latency processing near the data source.
- Primary constraints: Limited energy consumption, strict memory limits, and constrained processing power.

Comparative Overview

Aspect	Cloud / Remote	Local / On-site
Data Location	Remote processing	On-site local execution
Resource Availability	Virtually unlimited compute	Strictly resource-constrained
Response Time	Network-dependent latency	Deterministic real-time control
Primary Goal	Maximising throughput	Ensuring system reliability and low-latency

Private Data Centres: The On-Premises Era

Characteristics of Local Infrastructure

- **On-Premises Infrastructure:** Dedicated server racks owned, housed, and managed locally by the organisation.
- **Complete Operational Control:** Direct control over physical security, hardware selection, and network topology.
- **Isolated Environment:** Tailored compliance, strict data governance, and predictable local network traffic.

Private Data Centres: Core Limitations

Financial & Operational Bottlenecks

- **High Capital Expenditure (CapEx):** Substantial upfront investment in servers, storage arrays, power supply, and HVAC cooling.
- **Rigid Over-Provisioning:** Systems sized for peak capacity, resulting in severe resource under-utilisation during normal operation.
- **High Operational Overhead:** Slow procurement cycles, manual OS patching, physical maintenance, and complex disaster recovery.

The Stepping Stone: Virtualisation & Hosting

Bridging Bare-Metal to Cloud

- **Hardware Abstraction:** Hypervisors enabled multiple Virtual Machines (VMs) on a single physical host, improving density.
- **Colocation & Managed Hosting:** Shift from housing physical servers on-site to renting space, power, and bandwidth in shared data centres.
- **Remaining Friction:** Provisioning was still largely static, requiring manual intervention and long-term contract commitments.

The Cloud Computing Paradigm

NIST Definition & Core Essential Characteristics

A model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources:

- **On-Demand Self-Service:** Provision compute and storage without human interaction from service providers.
- **Broad Network Access:** Capabilities accessible over the network through standard mechanisms.
- **Resource Pooling & Elasticity:** Dynamic allocation of multi-tenant resources scaling rapidly with demand.

Cloud Architecture: Service Model

- **IaaS (Infrastructure as a Service):** Raw compute, storage, and networking (e.g., VMs, VPCs).
- **PaaS (Platform as a Service):** Managed runtime environments and middleware for developers.
- **SaaS (Software as a Service):** Fully managed end-user software applications.

Cloud Architecture: Deployment Model

- **Public Cloud:** Multi-tenant infrastructure owned by hyperscalers.
- **Private Cloud:** Dedicated cloud infrastructure for a single organisation.
- **Hybrid / Multi-Cloud:** Integrated public and private environments.

Cloud Value Proposition

- **Financial Shift (CapEx to OpEx):** Eliminates heavy upfront infrastructure costs in favour of flexible pay-as-you-go pricing.
- **Elasticity & Global Scale:** Instantaneous scaling up or down to match real-time application traffic globally.
- **Offloaded Management:** Maintenance, physical hardware upgrades, and infrastructure redundancy managed by hyperscalers.

Limitations of Centralised Cloud

Bottlenecks in Modern Workloads

- **WAN Latency & Jitter:** High Round Trip Times (100 ms+) render central clouds unsuitable for real-time actuators and low-latency feedback loops.
- **Bandwidth Bottlenecks:** Unfiltered streaming of raw video and telemetry causes network saturation and expensive data egress charges.
- **Connectivity Dependency:** Total loss of operational capability when WAN connectivity is interrupted.

Architectural Evolution: Comparative Overview

Dimension	Private Data Centre	Virtualised / Hosted	Centralised Cloud
Cost Model	High CapEx	Mixed CapEx/OpEx	Pure OpEx (Pay-as-you-go)
Provisioning	Months (Hardware)	Days/Hours (VMs)	Seconds (On-demand)
Scalability	Rigid / Over-provisioned	Semi-elastic	Virtually Unlimited
Control	Full Physical Control	Hardware Shared	Provider Managed
Latency	Low Local RTT	Variable	High WAN RTT (100 ms+)

Emergence of Edge Computing

- **Physical-Logical Proximity.** Executing algorithms directly on sensors, gateways, or local controllers, minimising the physical distance that data travels.
- **Deterministic Response Time.** Guaranteeing predictable latency on the order of milliseconds, essential for closed-loop control.
- **Source Filtering.** Massive reduction in transmitted data volume, sending only relevant metrics and alerts to central systems.

Concept of Fog Computing

The Intermediate Layer (Cisco / OpenFog)

Introduced to bridge the gap between Edge devices and the central Cloud.

Fog Computing extends compute, storage, and networking resources along the entire path between devices and traditional Data Centres.

Regional Aggregation & Analysis

Operates through regional nodes (Gateways, smart Routers, Micro Data Centres).

Ideal for cross-site data aggregation, neighbourhood load balancing, and distributed coordination.

The Cloud-Fog-Edge Continuum

Hierarchical Orchestration

The Continuum does not replace the Cloud, but integrates it into a fluid, multi-tiered architecture:

1. Edge

Capture, signal conversion, and immediate real-time reaction at the device level.

2. Fog

Regional filtering, sensor fusion, and structured intermediate processing.

3. Cloud

Big Data analytics, AI model training, and long-term historical archiving.

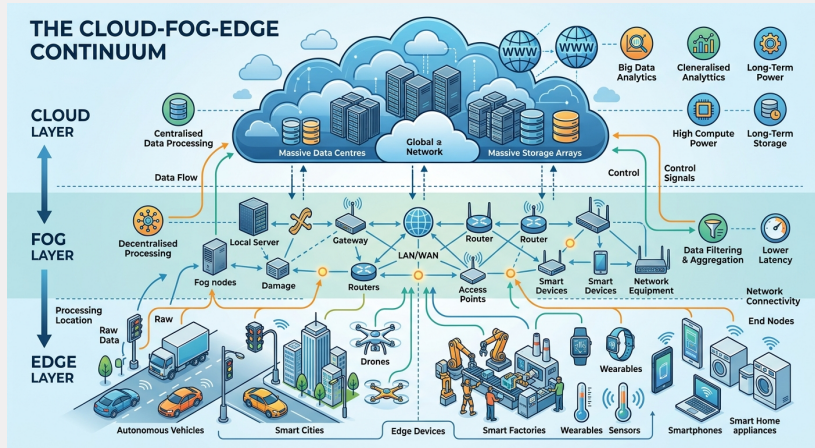
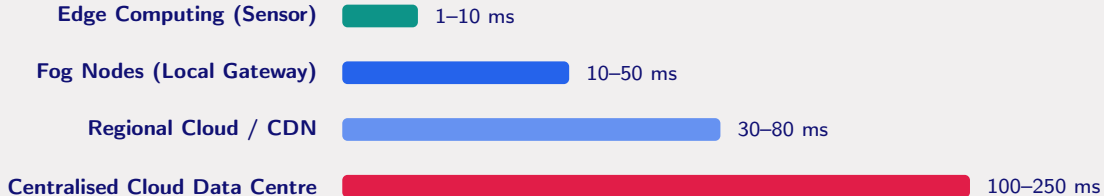


Figure: Overview of the Cloud-Fog-Edge Architecture (generated by Gemini AI, Google).

Response Latency Comparison



Data Sovereignty and Privacy

- **On-Premises Retention.** Maintaining confidential industrial data and intellectual property within physical company boundaries, avoiding public cloud exposure.
- **GDPR Compliance.** Strict compliance with regulatory standards such as the GDPR, ensuring sensitive personal data is anonymised or processed locally.
- **Risk Minimisation** Only aggregated telemetry or de-identified metrics are exported to the WAN, substantially reducing the overall attack surface.

Resilience and Autonomous Operation

- **Disconnected Operation.** Edge nodes continue executing local control routines and critical decisions even during complete WAN connectivity outages.
- **Local Buffering.** Utilising local memory and storage to temporarily retain state and events during network disruption periods.
- **Eventual Sync.** Intelligent reconnection with automatic delta synchronisation and subsequent state reconciliation with central cloud infrastructure.

Computational Resource Trade-offs

Dimension	Edge Devices	Fog Nodes	Cloud Data Centres
Power Consumption	Very Low (watts)	Moderate (hundreds of watts)	Massive (megawatts)
Space Constraints	Very High (Form factor)	Medium (Factory Racks)	Unconstrained (Dedicated)
Compute Capacity	Limited	Intermediate	Elastically Unlimited
Cooling	Passive / Aircooled	Standard local AC	Complex industrial HVAC

Exemplar Applications

- **Autonomous Vehicles.** Local processing of LIDAR and camera feeds for sub-millisecond braking and steering decisions (V2X).
- **Industry 4.0.** Deterministic control of robotic arms, computer vision quality inspection, and predictive maintenance.
- **Smart Grids.** Continuous balance between renewable generation and consumption via microgrids and real-time response.

Virtualisation and Containerisation

The Concept of Virtualisation

- **Resource Abstraction:** Decoupling physical hardware from the software layers running above it.
- **Logical Pools:** Transforming physical CPU, memory, storage, and networking into flexible logical resources.
- **Consolidation:** Allowing multiple independent workloads to safely run on a single physical machine.

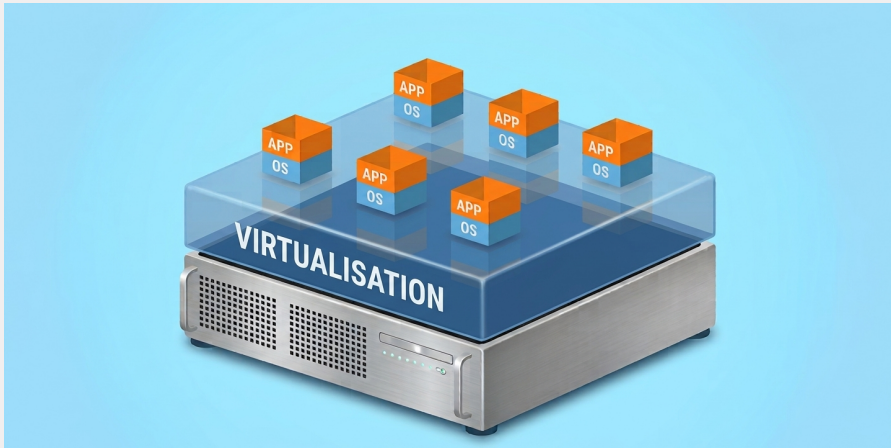


Figure: Concept of Virtualisation (generated by Gemini AI, Google).

Type-1 Hypervisors (Bare-Metal)

- **Definition:** A hypervisor that runs directly on the host's hardware to manage guest operating systems.
- **Architecture:** Deployed directly on the physical hardware without an underlying host operating system.
- **Key Characteristics:**
 - ▶ Direct access to hardware resources, delivering near-native performance.
 - ▶ Minimal overhead and enhanced security due to a reduced attack surface.
 - ▶ Enterprise-grade scalability, high availability, and robust resource isolation.
- **Primary Use Cases:** Data centres, cloud infrastructure, and mission-critical enterprise workloads.
- **Examples:** VMware ESXi, Proxmox VE, Microsoft Hyper-V, KVM.

Type-1 Hypervisor Architecture and Operation

- **Kernel-Level Management:**

- ▶ The virtualization software runs natively within the hypervisor's own kernel space.
- ▶ Schedules CPUs, allocates memory, and manages I/O directly without a host OS layer.

- **Microkernel vs. Monolithic Approach:**

- ▶ Stripped-down microkernel designs minimize overhead and security vulnerabilities.
- ▶ Unlike general-purpose kernels (e.g., Linux), non-essential features are omitted (e.g., full desktop stacks, generic user applications, peripheral drivers).

- **Hardware-Assisted Virtualization:**

- ▶ Leverages CPU extensions (Intel VT-x, AMD-V) to run guest OS kernels at root privilege safely (Ring -1 execution).

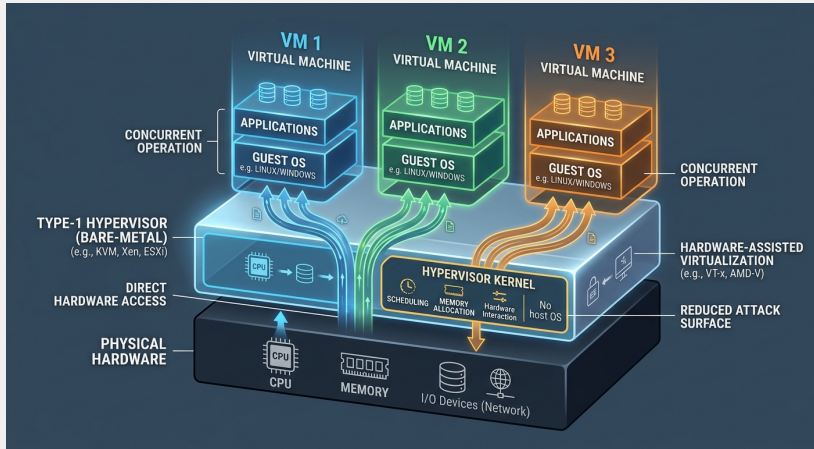


Figure: Concept of Type-1 Hypervisor (generated by Gemini AI, Google).

Type-2 Hypervisors (Hosted)

- **Architecture:** Runs as a software application on top of a conventional Host Operating System (e.g., Windows, macOS, Linux).
- **Key Characteristics:**
 - ▶ Relies on the host OS for hardware management, device drivers, and scheduling.
 - ▶ Introduces additional performance overhead due to OS-level abstractions.
 - ▶ Exceptional convenience, broad hardware support, and rapid deployment.
- **Primary Use Cases:** Local development environments, desktop virtualization, and software testing.
- **Examples:** Oracle VM VirtualBox, VMware Workstation, Parallels Desktop.

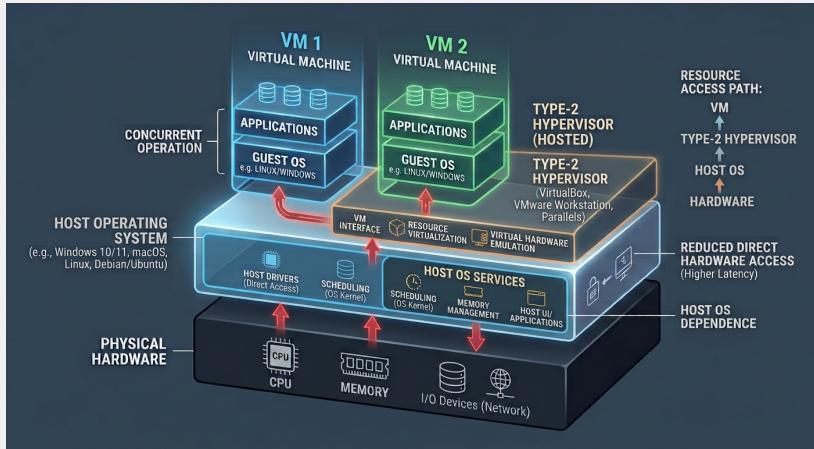


Figure: Concept of Type-2 Hypervisor (generated by Gemini AI, Google).

Overhead Costs in Full Virtualisation

- **Guest OS Duplication:** Every Virtual Machine must load its own entire kernel and system libraries.
- **Fixed RAM Consumption:** RAM allocated to a VM is permanently locked and unavailable to the host.
- **I/O Latency:** Disk and network requests must pass through multiple translation layers.

Evolution of Linux Containers

- **The chroot Genesis:** Introduced the primitive concept of altering the root directory for a process.
- **Process Isolation:** Evolved from simple file system locking to full operational isolation.
- **Linux Containers (LXC):** Combined various kernel primitives to create the first robust container ecosystem.

Shared Linux Kernel Anatomy

- **Single Kernel Principle:** Unlike VMs, all containers share the exact same underlying host kernel.
- **Isolated Processes:** Containers are merely standard Linux processes with restricted visibility.
- **Resource Elimination:** No guest kernels to boot, reducing architectural duplication to zero.

Linux Namespaces (Part 1)

- **PID Namespace:** Allocates an isolated process tree where the container starts at PID 1.
- **NET Namespace:** Provides independent network stacks, virtual interfaces, and firewall rules.
- **MNT Namespace:** Isolates file system mount points, hiding the host layout from the container.

Linux Namespaces (Part 2)

- **IPC Namespace:** Restricts Inter-Process Communication to processes inside the same namespace.
- **UTS Namespace:** Allows independent hostnames and domain names per container.
- **USER Namespace:** Maps UIDs/GIDs, ensuring that root privileges inside a container mean nothing on the host system.

Control Groups (cgroups v1/v2)

- **Resource Metering:** Measures and limits how much a process or container can consume.
- **CPU Throttling:** Controls shares and hard quotas to prevent a single process from hijacking the CPU.
- **Memory Enforcements:** Sets strict RAM boundaries, triggering OOM (Out Of Memory) kills if breached.
- **Block I/O Limits:** Throttles read/write speeds (IOPS) to protect storage performance.

Security Mechanisms

- **Seccomp Profiles:** Filters and blocks specific system calls (*syscalls*) to minimise the kernel attack surface.
- **LSM (AppArmor / SELinux):** Enforces mandatory access control policies on files, capabilities, and network ports.
- **Privilege Reduction:** Drops unnecessary Linux capabilities by default to secure the runtime environment.

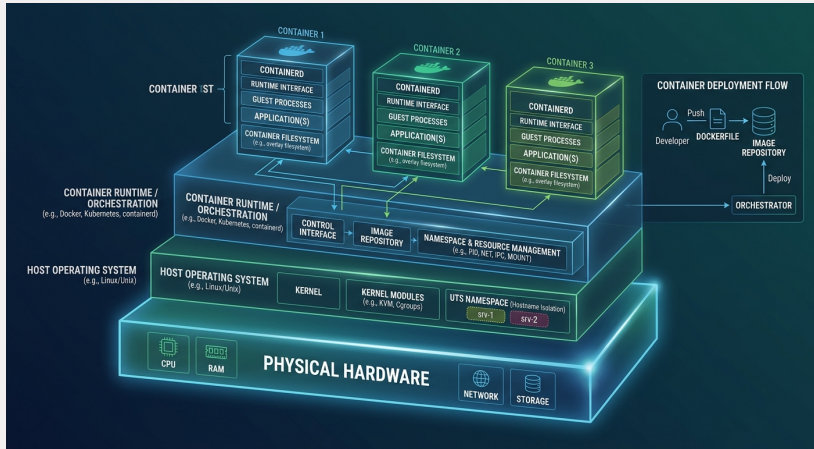


Figure: Concept of Application Containers (generated by Gemini AI, Google).

System Containers vs Application Containers

- **System Containers (LXC):**

- ▶ Act like lightweight VMs running a full init system (systemd).
- ▶ Designed to host multiple persistent services.

- **Application Containers (Docker):**

- ▶ Designed to wrap and run a single primary process or microservice.
- ▶ Ephemeral lifecycle based on immutable infrastructure principles.

Metric Comparison

Metric	Virtualisation (VM)	Containerisation
Instance Density	Low to Medium (Tens)	Very High (Hundreds)
Boot Time	Seconds to Minutes	Milliseconds to Seconds
CPU/RAM Efficiency	Higher overhead (Loss)	Near-native efficiency
Storage Footprint	Gigabytes (GB)	Megabytes (MB)

Proxmox VE: Architectural Foundations

- **Debian-Based Core:** Built on a bare-metal Debian Linux distribution, deploying a customized kernel with built-in virtualization modules.
- **Type-1 Hypervisor Design:** Integrates the **KVM** kernel module to convert the host OS directly into a high-performance bare-metal hypervisor.
- **Dual Engine Support:**
 - ▶ **KVM/QEMU:** Provides full virtualization for running unmodified guest OSs (Windows, Linux, BSD).
 - ▶ **LXC (Linux Containers):** Provides lightweight OS-level virtualization sharing the host kernel for lower overhead.

Proxmox VE: Component Roles and Responsibilities

- **KVM (Kernel Space):**
 - ▶ Direct CPU scheduling and memory allocation.
 - ▶ Leverages Intel VT-x / AMD-V extensions for near-native VM execution.
- **QEMU (User Space):**
 - ▶ Emulates virtual hardware (BIOS, PCI buses, virtual disk controllers, NICs).
 - ▶ Intercepts I/O operations and passes them efficiently to the host storage/network stack.
- **PVE Management Layer:**
 - ▶ Runs web GUI, REST API, and cluster daemons (pvedaemon, corosync) outside the hot execution path.

Proxmox VE: Data Flow and Execution Stack

- **Execution Layers:**
 - ▶ **Layer 4 (Top):** Guest VMs (Applications + Guest OS).
 - ▶ **Layer 3:** QEMU Process (Hardware Emulation & Device Mapping).
 - ▶ **Layer 2:** Linux Kernel + KVM Module (Privileged CPU/Memory Access).
 - ▶ **Layer 1 (Bottom):** Physical Hardware (Processor, RAM, Storage, Network).
- Management components never sit between the VM and CPU/RAM execution, ensuring enterprise-grade performance.